# O que falta (fonte única) — backlog operacional e critérios de "v1 vendável"

- **Última atualização**: Fev/2026 (06/02)

## Entregue em 07/02/2026 (resumo executivo)

- `event_ledger -> runtime`: falha estrita para webhooks/flows com retry/backoff/DLQ efetivos via `execution_jobs`.
- Observabilidade operacional: métricas de dispatch por target (`webhooks/flows/combined`) com `retryingPending`, `dead`, `deadRecent`.
- PBX call events -> runtime: idempotência por `eventId` (sem colapsar `ringing/answered/ended` com mesmo `correlationId`).
- Selftest operacional por domínio: check `pbxEventRuntime` em `/health/selftest` com limiares por ambiente (`dev/stage/prod`).

Este documento é a **fonte de verdade única** do "o que falta" a partir do estado atual do repositório (Fev/2026).
Ele substitui a visão antiga "capability-driven vs benchmarks" como lista operacional de execução.

> Referências: `docs/ROADMAP_3A5_ANOS.md` (Now/Next/Later) e `docs/O_QUE_FALTA_GAPS_VS_BENCHMARKS.md` (histórico/benchmark). Relatório: `docs/O_QUE_FALTA_DIVERGENCIAS.md` (rotas declaradas vs codebase). UI por épico: `docs/O_QUE_FALTA_EPICO_PAGES.md`. Checklist de UI: `docs/O_QUE_FALTA_UI_CHECKLIST.md`.

---

## Resumo por épico

Seções ordenadas por maior número de rotas faltando.

- P2.5 Lembretes e tarefas: 0 faltando / 4 total
- P0.1 Inbox multiatendente (lista, assumir, transferir, tags, notas, respostas rápidas): 0 faltando / 9 total
- P0.2 Painel supervisor com métricas (TME, TMA, abandono, por agente/fila/período): 0 faltando / 3 total
- P0.3 RBAC completo (admin/supervisor/agente): 0 faltando / 4 total
- P0.4 MFA funcional: 0 faltando / 4 total
- P0.5 Auditoria de ações: 0 faltando / 2 total
- P0.6 Mascaramento de telefone: 0 faltando / 2 total
- P0.7 Wizard de onboarding (WhatsApp, filas, usuários): 0 faltando / 5 total
- P1.1 Widgets + links rastreáveis (source/campaign): 0 faltando / 4 total
- P1.2 URA WhatsApp empacotada (UI simples, departamentos): 0 faltando / 4 total

- P1.3 Conexão WhatsApp via QR (além da Cloud API): 0 faltando / 5 total
- P2.1 IA operacional (resumo, sugestão, intenção, sentimento): 0 faltando / 5 total
- P2.2 API pública com API Keys + rate limit: 0 faltando / 4 total
- P2.3 Automações low-code: 0 faltando / 4 total
- P2.4 SLA por cliente: 0 faltando / 3 total

Checklist gerado automaticamente a partir de `docs/O_QUE_FALTA.md`.

## Pendências confirmadas no frontend (repo)

Itens abaixo foram verificados no `frontend/src/pages` e **não possuem página/UI dedicada** no repo atual.

- (nenhum)

Itens com UI já presente no repo:

- Suites verticais (MVP→v1 operacional)
- Marketing (v1 completo)
- SaaS e governança (base enterprise já entregue)
- Analytics e marketplace (primeiro corte)
- Governança enterprise (primeiro corte)
- P0.1 Inbox multiatendente (lista, assumir, transferir, tags, notas, respostas rápidas)
- P0.5 Auditoria de ações
- P0.7 Wizard de onboarding (WhatsApp, filas, usuários)
- P1.3 Conexão WhatsApp via QR (além da Cloud API)
- P2.1 IA operacional (resumo, sugestão, intenção, sentimento)
- P2.2 API pública com API Keys + rate limit
- P2.4 SLA por cliente
- P2.3 Automações low-code
- P2.5 Lembretes e tarefas

## 1) Estado atual (Fev/2026) — o que já foi entregue (resumo)

**Core (eventos + execução)**

- **Event Model canônico v1**: `api/events/eventModelV1.js`
- **Event ledger append-only (`event_ledger`)**: migration `api/db/migrations/20260204020000-create`
  + `api/services/eventLedgerService.js`
- **Retenção mínima (prune)**: `api/services/pruneEventLedgerService.js`
  (por `EVENT_LEDGER_RETENTION_DAYS`)
- **Execution Queue + Scheduler v1 (DB-first)**:
  - migrations `api/db/migrations/20260204021000-create-exec-jobs-and-schedules.js`
  - worker `api/services/executionQueueService.js`

  - scheduler `api/services/schedulerService.js`
  - SLA scan `api/services/opsSlaService.js` (`DeliverySLARisk`, `PhysicalSLARisk`)
- **Execution ledger por nó (v1, DB-first)**:
  - migrations `api/db/migrations/20260204031000-create-flow-execution-ledger.js` (tabelas `flow_executions` + `node_executions`)
  - serviço: `api/services/flowExecutionLedgerService.js` (header + steps por node; replay controlado em modo simulation)
- **Dispatcher de triggers a partir do `event_ledger` (v1 parcial)**:
  - cursor: migration `api/db/migrations/20260204033000-event-dispatch-state-and-schedule.j` (`event_dispatch_state` + schedule `scan_event_triggers`)
  - serviço: `api/services/eventLedgerTriggerDispatcher.js` (scan + enqueue + dispatch via webhooks)
  - extensão do estado: migration `api/db/migrations/20260204034000-event-dispatch-state-add-`
  - trigger de flows com modo estrito (retry/DLQ por falha): `api/services/flowEventTriggerService.js` (mapeamento via `metadata.eventTriggers`)

**Suites verticais (MVP→v1 operacional)**

- **Fast service (filas físicas)**:
  - Mudanças frontend: UI existente em `KioskPage.jsx`, `PanelPage.jsx`, `StationPage.jsx` (cobertura parcial; validar fluxo completo).
  - Falta na UI: painel admin de filas (CRUD, multi-unidade), métricas completas por unidade/estação e auditoria operacional.
  - público por token: `/physical/*` (kiosk/panel/station) em `api/server.js`
  - operações adicionais (station): `start/skip/transfer`
  - admin: `GET/POST /api/physical/queues`, `GET /api/physical/metrics`
  - sequenciamento robusto: `physical_ticket_sequences` (migration `20260204022000-physical-queues-multiunit-and-sequences.js`)
- **Pedidos/Storefront/Delivery**:
  - Mudanças frontend: UI existente em `StorefrontOrderPage.jsx`, `StorefrontOrderStatusPage.jsx`, `OrdersPage.jsx`, `KitchenPage.jsx`, `DispatchPage.jsx` (cobertura parcial; validar fluxo completo).
  - Falta na UI: dashboards operacionais (KPIs order_to_ready/order_to_deliver), incidentes com trilha auditável e filtros avançados.
  - público por token: `POST /storefront/orders`
  - hardening v1: idempotência por `idempotencyKey` (storefront) + propagação de `correlationId` (best-effort)
  - métricas: `GET /api/orders/metrics` (agregações por `locationId`)
  - incidente mínimo: `POST /orders/:id/incidents` (abre ticket digital e emite evento)
  - incidente operacional (omnichannel): ticket 1:1 com o pedido, idempotência e status
- **Omnichannel v1**:

  - unificação conversa ticket 1:1 com idempotência
  - status por mensagem (delivered/read/failed) na operação
  - roteamento/skills básicos, SLA simples e histórico unificado
- **Templates "solutions" por segmento**:
  - pacotes E2E (fastfood/pizzaria/clínica) com operação e métricas mínimas
- **PBX v1**:
  - Mudanças frontend: UI existente em `PbxPage.jsx` (cobertura parcial; validar fluxo completo).
  - Falta na UI: cobertura completa de provisionamento (tokens/expiração), auditoria e operações avançadas.
  - base produto/realtime/CDR via migrations PBX
  - `POST /api/pbx/dialplan/apply` com trilha mínima em `pbx_config_versions` (signature)
  - **Provisionamento de ramais (MVP)**: `POST /api/pbx/extensions` aceita `provisioning` (vendor+mac) e expõe `GET /provision/<tenantKey>/<vendor>/<mac>` (ou com token). Ver `docs/PBX_PROVISIONING_GRANDSTREAM_AVAYA.md`.
  - **Ops v1.1**: rotação de token e provisionamento em lote no admin (frontend).
  - **Call events v0 (estado + schedule)**: migration `api/db/migrations/20260204035000-pbx-call` + `api/services/pbx/pbxCallEventsService.js`.

**Marketing (v1 completo)**

- Mudanças frontend: UI existente em `MarketingPage.jsx` (cobertura parcial; validar fluxo completo).
- Falta na UI: jornadas/funil visual completo e gestão avançada de campanhas.
- **Migrations e modelos base**: `api/db/migrations/20260204040000-marketing-v1-core.js` + models `Marketing*` em `api/db/models/`.
- **Serviço inicial**: `api/services/marketing/marketingService.js`.
- **Público**: `GET /mkt/track.js`, `POST /mkt/e`, `GET /lp/:slug`, `POST /lp/:slug/submit`.
- **Campanhas de email**: `POST /api/marketing/email-campaigns/:id/send` (envio best-effort).
- **UI/admin completa**: gestão de forms/leads/scoring/email + jornadas v1 e funil visual básico.

**Voz (base técnica entregue)**

- **Voice nodes v0 (engine)**: `engine/nodes/voiceNodes.js` + `engine/__tests__/voiceNodes.test.js`.
- **Ingestão básica de eventos de voz**: `api/services/voiceEventService.js` + `POST /api/voice/events`.

**RPA Flow Pipeline (base entregue)**

- **Serviço**: `api/services/rpaFlowPipelineService.js` (análise → seeds → validação → conversão flow→RPA).
- **Endpoints**: preview e execução no `api/server.js`.
- **Playbook operacional**: `docs/RPA_FLOW_PIPELINE_PLAYBOOK.md`.

**SaaS e governança (base enterprise já entregue)**

- Mudanças frontend: UI existente em `TenantsPage.jsx`, `SuperAdminPage.jsx`, `PlanUsagePage.jsx`, `SsoScimPage.jsx`, `BackupPage.jsx` (cobertura parcial; validar fluxo completo).
- Falta na UI: governança granular (roles/permissões detalhadas) e dashboards completos de uso por tenant.
- **Multi-tenant + RBAC + planos/limites + auditoria + super-admin + onboarding + rate-limit por tenant**: ver `IMPLEMENTACAO_SAAS_COMPLETA.md`.

**Hardening de IA (contratos + safe-mode)**

- **AI Output Envelope (obrigatório)**:
  - especificação: `docs/IA_OUTPUT_ENVELOPE.md`
  - schema: `contracts/ia/ai-output-envelope.schema.json`
  - implementação/testes: `engine/ai/aiEnvelope.js` e `tests/unit/aiEnvelope.test.js`

**Integrações de plataforma (base)**

- **Webhooks customizados**: `api/services/webhookService.js`
- **WebSockets (real-time por tenant)**: `api/services/websocketService.js`
- **Validação de inputs**: `api/utils/inputValidator.js` (resumo em `IMPLEMENTACOES_MELHORIAS.md`)

**Analytics e marketplace (primeiro corte)**

- Mudanças frontend: UI existente em `AnalyticsPage.jsx`, `NodeMarketplacePage.jsx` (cobertura parcial; validar fluxo completo).
- Falta na UI: atribuição/ROI avançado, custos operacionais por canal e dashboards completos por suíte.
- **Analytics/ROI v0**: `api/services/roiService.js` + `GET /analytics/roi` + UI em `frontend/src/pages/AnalyticsPage.jsx`.
- **Node marketplace v0** (catálogo estático): `api/services/nodeMarketplaceService.js` + `GET /api/node-marketplace/catalog` + UI `frontend/src/pages/NodeMarketplacePage.jsx`.
  - **Próximo foco**: evoluir para **Analytics/ROI v1** (funil básico, custos por canal e KPIs de SLA).

**Governança enterprise (primeiro corte)**

- Mudanças frontend: UI existente em `SsoScimPage.jsx`, `BackupPage.jsx` (cobertura total; validar fluxo completo).
- **Ambientes de flow (dev/stage/prod)**: `GET /flows?environment=...` + `PUT /flows/:id/environment` + seletor no editor.
- **SSO/SCIM (config por tenant)**: `api/services/tenantSettingsService.js` + endpoints `/api/tenant-settings/:key` + UI `frontend/src/pages/SsoScimPage.jsx`.
- **Backup/DR básico**: `GET /api/backup/export` + `POST /api/backup/import` + UI `frontend/src/pages/BackupPage.jsx`.

**Gate operacional**

- **Selftest**: `GET /health/selftest`
- **Runner de stack**: `scripts/system-autotest-stack.sh` com gate de selftest (FAIL bloqueia)
- **Unit tests**: `npm run test:unit` passando no estado atual
- **Bootstrap/autoprovisionamento (base)**:
  - `startInitialization()` em `api/server.js` garante `systemadmin`/tenant `system` (primeira inicialização).
  - Seed idempotente: `api/db/seed.js` (+ utilitário `scripts/create-super-admin.js`).
  - One-click bootstrap (super-admin): `POST /super-admin/bootstrap/one-click` (serviço `api/services/oneClickBootstrapService.js`).

**Entregas concluídas (ex-backlog)**

- **Execution ledger**: endpoints de consulta (execution_id/correlation_id/subject_id) com RBAC e replay parcial "from node" com trilha auditável.
- **Queue/Scheduler enterprise mínimo**: quotas por tenant, DLQ consultável/reprocessável, métricas (lag/falhas/retries) e selftest por limiar.
- **Triggers do `event_ledger` (v1)**: mapping evento→flow/trigger com idempotência; dispatcher v1 para webhooks e flows, com falha estrita (retry/DLQ) e métricas por target.
- **PBX call events v0**: ringing/answered/ended no `event_ledger`, correlação com ticket/pedido e base DB-first (estado + schedule).
- **Fast service v1**: auditoria operacional consultável, métricas completas e contratos de estados com idempotência.
- **Orders/Delivery v1**: KPIs `order_to_ready`/`order_to_deliver` + alertas e dashboard mínimo.
- **Tokens públicos**: endpoints admin (listar/revogar/rotacionar), rate-limit por token/kind e dashboards mínimos de uso.
- **RPA Flow Pipeline v1.1**: documentação/playbooks, testes E2E e validações de segurança, observabilidade (logs/métricas/audit).
- **Autoprovisionamento v1**: checklist/relatório por step, opções por suíte/unidade e gates de segurança.
- **PBX provisioning v1.1**: templates vendor/model/firmware, parâmetros operacionais, rotação/expiração de token com trilha auditável e bulk

+ sanity check.

- **Core DAG + event model (consolidação)**: triggers/handlers full-fidelity do `event_ledger` e retenção por tenant com custo previsível.
- **Plataforma integrada**: SDK/marketplace de nodes, Analytics/ROI completo e Voz v1 no DAG.
- **Enterprise**: HA/DR, auditoria imutável, programa de parceiros e marketplace monetizado.

---

## 2.1) Roadmap competitivo (ChatPro) — P0/P1/P2

Backlog competitivo focado em lacunas de operação CCaaS. Rotas marcadas como presentes/ausentes foram validadas contra o codebase.

### P0.1 Inbox multiatendente (lista, assumir, transferir, tags, notas, respostas rápidas)

- Status (auto): backend ok | frontend ok
- Mudanças frontend: UI dedicada em `InboxPage.jsx` + filtros e SLA; fluxo de apoio segue em `SupportPage.jsx`, `QuickRepliesPage.jsx`, `TicketMaintenancePage.jsx`.
- Falta na UI: —
- Rotas backend:
  - ☒ GET /quick-replies
  - ☒ POST /quick-replies
  - ☒ POST /tickets/:id/tags
  - ☒ DELETE /tickets/:id/tags/:tag
  - ☒ GET /inbox/conversations
  - ☒ GET /inbox/conversations/:id
  - ☒ POST /inbox/conversations/:id/assign
  - ☒ POST /inbox/conversations/:id/transfer
  - ☒ POST /inbox/conversations/:id/notes

### P0.2 Painel supervisor com métricas (TME, TMA, abandono, por agente/fila/período)

- Status (auto): backend ok | frontend ok
- Mudanças frontend: UI dedicada em `SupervisorPage.jsx` com filtros e tabelas de status.
- Falta na UI: exportação dedicada e gráficos avançados (opcional).
- Rotas backend:
  - ☒ GET /supervisor/metrics?groupBy=agent|queue&period=day|hour&from=…&to=…
  - ☒ GET /supervisor/agents/status
  - ☒ GET /supervisor/queues/status

### P0.3 RBAC completo (admin/supervisor/agente)

- Status (auto): backend ok | frontend ok
- Mudanças frontend: UI dedicada em `RbacPage.jsx` com criação de role, lista de permissões e atribuição.
- Falta na UI: guardas adicionais por módulo/ação (opcional).
- Rotas backend:
  - ☒ GET /rbac/roles
  - ☒ POST /rbac/roles
  - ☒ GET /rbac/permissions
  - ☒ POST /rbac/assign

### P0.4 MFA funcional

- Status (auto): backend ok | frontend ok
- Mudanças frontend: UI dedicada em `MfaPage.jsx` (setup, verify, disable e recovery).
- Falta na UI: QR code visual (opcional).
- Rotas backend:
  - ☒ POST /auth/mfa/setup
  - ☒ POST /auth/mfa/verify
  - ☒ POST /auth/mfa/disable
  - ☒ POST /auth/mfa/recovery

### P0.5 Auditoria de ações

- Status (auto): backend 2/2 | frontend ok
- Mudanças frontend: UI existente em `AuditPage.jsx` (cobertura total; validar fluxo completo).
- Rotas backend:
  - ☒ GET /audit/logs?resource=…&action=…&startDate=…&endDate=…
  - ☒ GET /audit/stats

### P0.6 Mascaramento de telefone

- Status (auto): backend ok | frontend ok
- Mudanças frontend: toggle em `ConfigPage.jsx` + componente `MaskedPhone` aplicado em Inbox/Support/TicketMaintenance.
- Falta na UI: —
- Rotas backend:
  - ☒ GET /api/tenant-settings/phone-masking
  - ☒ POST /api/tenant-settings/phone-masking

### P0.7 Wizard de onboarding (WhatsApp, filas, usuários)

- Status (auto): backend 5/5 | frontend ok

- Mudanças frontend: UI existente em `OnboardingPage.jsx` (cobertura total; validar fluxo completo).
- Rotas backend:
  - ☒ GET /onboarding/status
  - ☒ POST /onboarding/start
  - ☒ POST /onboarding/complete-step
  - ☒ POST /onboarding/complete
  - ☒ POST /onboarding/create-templates

### P1.1 Widgets + links rastreáveis (source/campaign)

- Status (auto): backend ok | frontend ok
- Mudanças frontend: gestão de links/UTM em `MarketingPage.jsx`.
- Falta na UI: painel de performance avançado (opcional).
- Rotas backend:
  - ☒ POST /api/marketing/links
  - ☒ GET /api/marketing/links
  - ☒ GET /mkt/l/:code
  - ☒ POST /mkt/click

### P1.2 URA WhatsApp empacotada (UI simples, departamentos)

- Status (auto): backend ok | frontend ok
- Mudanças frontend: UI dedicada em `WhatsAppIvrPage.jsx` (departamentos + regras simples).
- Falta na UI: wizard guiado e integrações avançadas (opcional).
- Rotas backend:
  - ☒ GET /whatsapp/ivr
  - ☒ POST /whatsapp/ivr
  - ☒ POST /whatsapp/ivr/department
  - ☒ POST /whatsapp/ivr/rule

### P1.3 Conexão WhatsApp via QR (além da Cloud API)

- Status (auto): backend ok | frontend ok
- Mudanças frontend: UI dedicada em `WhatsAppConnectionsPage.jsx` (pareamento, status e reconexão).
- Falta na UI: QR code visual (opcional).
- Rotas backend:
  - ☒ POST /whatsapp/connections
  - ☒ GET /whatsapp/connections
  - ☒ POST /whatsapp/connections/:id/qr
  - ☒ POST /whatsapp/connections/:id/disconnect
  - ☒ GET /whatsapp/connections/:id/status

### P2.1 IA operacional (resumo, sugestão, intenção, sentimento)

- Status (auto): backend 5/5 | frontend ok
- Mudanças frontend: UI existente em `SupportPage.jsx` (cobertura parcial; validar fluxo completo).
- Rotas backend:
  - ☒ GET /tickets/:id/ai-summary
  - ☒ POST /tickets/:id/ai-summary
  - ☒ GET /tickets/:id/ai-suggestions
  - ☒ GET /tickets/:id/ai-intent
  - ☒ GET /tickets/:id/ai-sentiment

### P2.2 API pública com API Keys + rate limit

- Status (auto): backend ok | frontend ok
- Mudanças frontend: UI dedicada em `DeveloperKeysPage.jsx` com criação e visão de uso.
- Falta na UI: relatórios avançados por rota e alertas (opcional).
- Rotas backend:
  - ☒ POST /developer/keys
  - ☒ GET /developer/keys
  - ☒ POST /public/*
  - ☒ GET /developer/usage

### P2.3 Automações low-code

- Status (auto): backend ok | frontend ok
- Mudanças frontend: UI dedicada em `AutomationsPage.jsx` (builder JSON + publicação/execução).
- Falta na UI: builder visual completo com drag-and-drop (opcional).
- Rotas backend:
  - ☒ GET /automations
  - ☒ POST /automations
  - ☒ POST /automations/:id/publish
  - ☒ POST /automations/:id/run

### P2.4 SLA por cliente

- Status (auto): backend ok | frontend ok
- Mudanças frontend: UI dedicada em `SlaPoliciesPage.jsx` (políticas + violações).
- Falta na UI: visão detalhada por cliente e alertas (opcional).
- Rotas backend:
  - ☒ GET /sla/policies
  - ☒ POST /sla/policies
  - ☒ GET /sla/breaches

**P2.5 Lembretes e tarefas**

- Status (auto): backend ok | frontend ok
- Mudanças frontend: UI dedicada em `TasksPage.jsx` (lista + conclusão + lembrete).
- Falta na UI: calendário e lembretes avançados (opcional).
- Rotas backend:
  - ☒ GET /tasks
  - ☒ POST /tasks
  - ☒ POST /tasks/:id/complete
  - ☒ POST /tasks/:id/reminder (opcional)

## 2) Backlog único (priorizado) — Now/Next/Later (o que falta)

> Regra: cada item abaixo é "único" (não duplicar em outros docs). Se um tema aparecer em mais de um pilar, ele deve ser referenciado (não replicado).

**Now (0–3 meses) — hardening para v1 vendável (suíte operável)**

**2.1 Execução "enterprise mínima" (core DAG)**

- **Conectar o execution ledger por nó ao runtime + expor consulta/replay assistido**
  - **Status**: base DB-first já existe (`flow_executions`/`node_executions` + `flowExecutionLedgerService`); integração full do runtime ainda parcial.
  - **Saídas mínimas**:
    - registrar execuções reais do DAG (não só simulation/flows específicos)
  - **Critério de pronto**: responder "o que aconteceu em qual node" e permitir replay controlado com rastreabilidade.

**2.2 Triggers do runtime a partir do `event_ledger`**

- **Consumer/dispatcher** de `event_ledger` para acionar execuções (regras mínimas)
  - **Status (07/02/2026)**: retry/backoff e DLQ integrados ao runtime/queue; falhas de dispatch propagam corretamente e entram em retry/DLQ.
  - **Status (07/02/2026)**: observabilidade ampliada com métricas de dispatch por target (`webhooks`/`flows`/`combined`) incluindo `retryingPending`, `dead`, `deadRecent`.
  - **Status (07/02/2026)**: selftest operacional por domínio entregue para PBX (`pbxEventRuntime`) com limiares por ambiente.

  - **Status (08/02/2026)**: selftest por domínio expandido para dispatch genérico do `event_ledger` (`eventDispatchRuntime`) com thresholds por backlog/retry/dead/lag e visão por target.

### 2.3 PBX: integração de call events v0 ao runtime

- acionar triggers/ações do DAG a partir dos eventos de chamada (ringing/answered/ended)
- **Status (07/02/2026)**: correlação/idempotência reforçadas no runtime para eventos PBX (dedupe por `eventId` quando há `correlationId` compartilhado entre `ringing/answered/ended`).
- **Status (07/02/2026)**: observabilidade operacional dedicada entregue via selftest `pbxEventRuntime` (scan + dispatch + volume recente de eventos PBX).

## Next (3–6 meses) — produto inicial por suíte + analytics/ROI v1

### 2.4 Analytics/ROI v1 (sair do v0)

- funil visual básico e atribuição mínima por canal
- custos por canal + KPIs de SLA mínimos (order_to_ready/order_to_deliver)
- dashboards operacionais por suíte (fast service + orders/delivery)
- **Status (08/02/2026)**: draft ROI v1 entregue na `AnalyticsPage` com funil visual, custos por canal, KPIs de SLA e dashboards operacionais por suíte (pedidos + filas físicas), incluindo filtro por `locationId`, recorte operacional por hora, visual de volume por hora, exportação CSV por hora, filtros avançados (faixa horária + métrica/status) aplicados à visualização/export, persistência dos filtros na URL (deep-link/share), ação de copiar link com filtros, presets operacionais de horário (dia completo/pico almoço/tarde/pico jantar) e cobertura dos blocos restantes com ações de export/reset (voz, tokens públicos e métricas de atendimento).

## Next (6–12 meses) — consolidação core + PBX/voz v1

### 2.5 PBX + voz v1 (base CCaaS)

- **Objetivo**: fechar o ciclo PBX/voz v1 com operação auditável ponta a ponta (configuração, runtime, observabilidade e custo).
- **Escopo funcional v1 (obrigatório)**:
  - URA visual v1 com versionamento de dialplan por tenant e trilha de publicação.
  - filas de voz v1 (membros, estratégia básica, overflow) com CRUD administrável.
  - gravação com trilha auditável (origem, consentimento/política, retenção, download protegido).
  - ingestão/runtime de eventos de voz (ringing/answered/ended + transfer/queue events) integrados ao Event Model.

  - observabilidade operacional de voz (volume, SLA básico, falhas/retries, health checks por domínio).
  - custos básicos de voz por tenant (chamadas, minutos, estimativa de custo por tipo/rota).
- **Status atual no repo (08/02/2026)**:
  - base PBX e provisioning existentes (`/api/pbx/*`, `POST /api/pbx/dialplan/apply`, provisionamento em `POST /api/pbx/extensions`).
  - call events PBX v0 e selftest operacional (`pbxEventRuntime`) já entregues.
  - base de runtime/ingestão de voz disponível (`POST /api/voice/events`, `/api/voice/metrics`, runtime service).
  - **Status (08/02/2026)**: selftest operacional de voz entregue (`voiceRuntime`) com thresholds por ambiente e visão de sessões ativas.
  - **lacuna principal**: empacotar tudo em fluxo produto-operação único (UI + contratos + QA + gate de release).
- **Blocos de entrega (2.5.x)**:
  - `2.5.1` URA visual + publicação controlada:
    - fluxo draft/publish/rollback por versão.
    - validação estrutural de fluxo de voz no publish (bloqueante).
    - auditoria: quem publicou, quando, diff e resultado do apply.
  - `2.5.2` Filas de voz operacionais:
    - CRUD completo (queues/members/routes) com validações de consistência.
    - visão operacional (fila atual, backlog, abandono, tempo médio de atendimento).
    - ações rápidas (ativar/desativar membro, dreno de fila, fallback de rota).
  - `2.5.3` Gravação e compliance:
    - política por tenant/unidade (on/off, retenção, mascaramento quando aplicável).
    - indexação de metadados (callId, tenant, data, duração, direção).
    - download/listagem com RBAC e auditoria.
  - `2.5.4` Runtime de voz no DAG:
    - gatilhos de voz para fluxos (com idempotência por `eventId`).
    - handoff voz atendimento digital com correlação única.
    - replay/auditoria de execução por call session.
  - `2.5.5` Observabilidade + custos:
    - painel de voz com KPIs (atendidas, perdidas, ASA básico, duração média, transferências).
    - selftest de voz com thresholds por ambiente (dev/stage/prod).
    - custo estimado por tipo de chamada e tendência diária.
- **Critério de pronto (DoD 2.5)**:
  - publicação de URA validada + rollback funcional sem intervenção manual.
  - fila de voz operável por UI com métricas mínimas e trilha auditável.

- gravações consultáveis com RBAC e retenção aplicada.
- eventos de voz acionando runtime com idempotência e rastreabilidade.
- dashboard de voz + selftest sem `FAIL` em stage/prod por 7 dias consecutivos.

---

## 3) Gates e "Definição de Pronto" (DoD) — para declarar v1 vendável

### 3.1 Gate mínimo de CI/Stack

- **Gate A — Integridade técnica (bloqueante)**:
  - `npm run test:unit` sem falhas.
  - `tools/system/validate.js` com `PASS`.
  - contrato/Event Model e schemas sem regressão.
- **Gate B — Fluxo operacional mínimo (bloqueante)**:
  - `scripts/system-autotest-stack.sh` cobrindo:
    - token público → pedido → kitchen/dispatch → status público.
    - token público → kiosk ticket → station (call/start/finish/skip/transfer) → panel refletindo.
    - PBX/voz: ingestão de evento + dispatch/runtime + selftest.
  - `/health/selftest` com resultado geral `PASS` ou `WARN` (qualquer `FAIL` bloqueia).
- **Gate C — Runtime/eventos (bloqueante)**:
  - checks mínimos obrigatórios no selftest:
    - `eventDispatchRuntime`
    - `pbxEventRuntime`
    - `voiceRuntime`
    - checks críticos de queue/scheduler/DB connectivity.
  - thresholds por ambiente devem estar configurados (`dev`, `stage`, `prod`) e versionados.
- **Gate D — Qualidade de release (bloqueante)**:
  - `scripts/autotest.js` com veredito `GO`.
  - sem migração pendente quebrando bootstrap.
  - sem endpoint crítico v1 sem cobertura mínima de teste.
- **Gate E — Operação e rollback (bloqueante)**:
  - playbook de rollback documentado por suíte crítica (orders/fast service/pbx).
  - evidência de restore/import para backup/DR básico.
- **Comando de verificação de release (sequência padrão)**:
  1. `npm run test:unit`
  2. `node tools/system/validate.js`
  3. `node scripts/autotest.js`
  4. `bash scripts/system-autotest-stack.sh`

5. `curl -s http://localhost:3000/health/selftest` (ou endpoint equivalente do ambiente)

### 3.2 DoD (por tema)

- **Ledger por nó + replay**:
  - execução auditável por `execution_id/node_id`.
  - replay parcial com trilha de origem/ator/timestamp.
  - retenção por tenant com política explícita.
- **Queue/scheduler enterprise mínimo**:
  - quotas por tenant ativas.
  - DLQ consultável e reprocessável.
  - métricas de lag/retry/failure + selftest por limiar.
- **PBX/voz runtime**:
  - idempotência por `eventId` preservada para `ringing/answered/ended`.
  - correlação única para trilha call→ticket/flow quando aplicável.
  - `pbxEventRuntime` saudável em produção sem `FAIL`.
- **Suites vendáveis (Fast service + Orders/Delivery)**:
  - contratos de estado consistentes + idempotência.
  - auditoria operacional consultável.
  - KPIs mínimos (`order_to_ready`, `order_to_deliver`, abandono/espera para físico).
- **Analytics/ROI v1**:
  - funil + custos por canal + KPIs operacionais.
  - dashboards por suíte com filtros/export/share.
  - consistência entre API e UI validada em testes.

---

## 4) Plano de implementação (tudo o que falta)

### 4.1 Princípios de execução

- manter bloco único de backlog: o que for concluído vira status nesta seção.
- evitar iniciativas paralelas sem gate técnico: cada onda fecha com validação.
- foco em "vendável e operável": só considerar pronto com teste + observabilidade + rollback.

### 4.2 Onda A (curto prazo) — fechar PBX/voz v1

1. `A-01` URA visual com publish/rollback + auditoria de publicação.
2. `A-02` filas de voz completas (CRUD + operação + métricas de fila).
3. `A-03` gravação com trilha auditável e política de retenção por tenant.
4. `A-04` runtime de voz no DAG com idempotência e correlação ponta a ponta.
5. `A-05` dashboard operacional de voz + custo básico por tenant.

6. A-06 selftest de voz por domínio com thresholds por ambiente.
7. A-07 suite de testes (unit/integration/e2e) para fluxo de voz e PBX.

### 4.3 Onda B (hardening de gates) — release disciplinado

8. B-01 consolidar scripts/system-autotest-stack.sh com cenários PBX/voz.
9. B-02 padronizar falha bloqueante (FAIL) no selftest em pipeline.
10. B-03 checklist de release com evidências (logs/artifacts) por gate.
11. B-04 playbooks de rollback por suíte crítica (orders/fast service/pbx).
12. B-05 cobertura de restore/import no fluxo de backup/DR básico.

### 4.4 Onda C (consolidação operacional)

13. C-01 Fast service: completar métricas por unidade/estação + trilha auditável.
14. C-02 Orders/Delivery: incidentes com trilha + alertas operacionais.
15. C-03 PBX admin: completar governança e operações avançadas na UI.
16. C-04 Analytics avançado: atribuição/custos por canal em nível de suíte.
17. C-05 documentação final por suíte (runbook, troubleshooting, SLO/SLI).

### 4.5 Definição de conclusão do bloco 4

- bloco 4 só é considerado completo quando:
  - todas as ondas A/B/C estão done.
  - gates A-E da seção 3 passam continuamente em stage.
  - existe evidência de rollout controlado e rollback funcional.
  - documentação operacional está atualizada e versionada.

### 4.6 Rastreio de status (atualizar a cada entrega)

- A-01: done (publish/rollback controlado de dialplan com historico em pbx_config_versions, endpoint POST /api/pbx/dialplan/rollback e trilha de auditoria pbx.dialplan.publish|rollback)
- A-02: done (filas de voz com CRUD+operacao e metricas operacionais via GET /api/pbx/queues/metrics + UI em PbxPage)
- A-03: done (politica por tenant em GET/POST /api/pbx/recordings/policy com retencao, mascaramento opcional e auditoria de download/configuracao)
- A-04: done (runtime de voz no DAG com idempotencia por eventId em api/services/voiceRuntimeService.js + correlacao por callId)
- A-05: done (dashboard operacional/custo de voz no AnalyticsPage consumindo GET /api/voice/metrics)
- A-06: done (voiceRuntime implementado em /health/selftest com thresholds por ambiente)
- A-07: done (suite de testes ampliada com voiceRuntimeService.idempotency, pbxQueuesService.metrics, selftestService.voiceRuntime e backup import/export)

- B-01: done (scripts/system-autotest-stack.sh consolidado com smoke de voz: scripts/test-voice-runtime-smoke.js)
- B-02: done (gate de release dedicado: tools/system/release-gate.js + npm run system:release-gate)
- B-03: done (artifact de evidência em qa/release-gate-report.json gerado pelo release gate)
- B-04: done (playbook documentado em docs/ROLLBACK_PLAYBOOK_SUITES_CRITICAS.md)
- B-05: done (cobertura de fluxo backup/restore API com teste tests/unit/backupImportExportEndpoints.test.js)
- C-01: done (fast service com metricas por unidade/estacao em physicalQueueService.getMetrics e trilha em event_ledger)
- C-02: done (orders/delivery com incidente rastreavel POST /orders/:id/incidents e metricas operacionais em /api/orders/metrics)
- C-03: done (PBX admin ampliado em PbxPage: historico de versoes, acao de rollback e governanca de politica de gravacao)
- C-04: done (analytics avancado por suite/canal consolidado em frontend/src/pages/AnalyticsPage.jsx + api/services/roiService.js)
- C-05: done (runbook/troubleshooting/SLO-SLI por suite em docs/RUNBOOK_SUITES_V1.md)

## 5) Proximo backlog real (pos-v1 vendavel)

Com os blocos A/B/C concluidos, o que "falta" agora e a agenda de evolucao (nao bloqueante para v1 vendavel).

### 5.1 UX e operacao (frontend)

- UX-01 Inbox/Support/TicketMaintenance: validacao automatizada de smoke entregue (scripts/test-inbox-runtime-smoke.js integrado ao system-autotest-stack.sh); manter validacao exploratoria periodica em stage para cenarios reais de handoff/SLA/mascaramento.
- UX-02 Supervisor: exportacao dedicada entregue na UI (SupervisorPage: CSV de metricas/agentes/filas) + visual operacional de carga (top filas/agentes); manter evolucao para graficos historicos avancados.
- UX-03 RBAC: guardas por modulo/acao entregues com validacao central de acesso por rota (canAccessPath em menuPermissions) aplicada no main.jsx, bloqueando bypass por URL direta com tela de acesso negado.
- UX-04 MFA e WhatsApp QR: exibicao de QR visual entregue em MfaPage e WhatsAppConnectionsPage (com fallback para texto/copia; no MFA, exibicao visual condicionada com aviso de privacidade por usar servico externo de imagem).
- UX-05 Automations: builder visual drag-and-drop entregue em primeiro corte no AutomationsPage (paleta de blocos, fluxo reordenavel por arraste, edicao de passo e sincronizacao bidirecional com JSON).
- UX-06 Tasks: calendario operacional mensal + agenda do dia e lembretes avancados entregues no TasksPage (atalhos 15m/1h/1d/no prazo sobre dueAt, mantendo agendamento manual).

- UX-07 SLA: visao detalhada por cliente + alertas proativos entregues no SlaPoliciesPage (agregacao de breaches por cliente, filtro operacional e alertas de pico/recorrencia/policies inativas).

### 5.2 Produto e analytics

- PRD-01 Marketing: funil/jornada visual avancados entregues no MarketingPage (funil com conversao entre estagios, tabela de jornada por origem/campanha e alertas operacionais de clique sem lead/baixa conversao).
- PRD-02 Developer Keys: relatorios por rota e alertas de consumo entregues (/developer/usage agora retorna agregacao routes via uso real de /public/*; DeveloperKeysPage exibe ranking por rota + alertas operacionais de pico/erro/expiracao).
- PRD-03 Analytics/ROI: atribuicao por canal + custos operacionais ponta a ponta entregues (API analytics/roi|funnel com custos operacionais/blended por canal e AnalyticsPage com painel de margem/ROI blended + alertas de eficiencia).
- PRD-04 PBX admin: operacoes avancadas de governanca entregues (/api/pbx/provisioning-token/revoke + status de expiracao e presets de rotacao no PbxPage + painel de auditoria PBX em /api/pbx/audit).

### 5.3 Governanca de release

- OPS-01 manter gate system:release-gate com evidencias em todas as releases.
- OPS-02 revisar thresholds de selftest por ambiente a cada mudanca de carga/perfil.
- OPS-03 simulados periodicos de rollback entregues (npm run system:rollback-drill com evidencias em qa/rollback-drill-report.json; modo com aplicacao controlada de rollback PBX via ROLLBACK_DRILL_APPLY_PBX=1).

### 5.4 Ordem sugerida (proxima onda)

1. UX-01, UX-02, UX-04 (maior impacto operacional imediato).
2. PRD-02, PRD-03 (impacto direto em monetizacao e gestao).
3. UX-05, UX-06, UX-07 (maturidade de produto e escala operacional).
4. PRD-01, PRD-04, OPS-03 (consolidacao e governanca de longo prazo).